Assignment 1 Report

> This report is written with the assistance of codex-5.3.

Task 01 Report (Monoalphabetic Substitution Cipher)

Recovered all 26 substitutions and decrypted meaningful English plaintext.

```
a -> c
b -> f
c -> m
...
y -> t
z -> u
```

```
the oscars turn on sunday which seems about right after this long strange awards trip..
```

Task 02 Report (OpenSSL enc with Multiple Ciphers)

Tested `-aes-128-cbc`, `-aes-128-cfb`, and `-aes-128-ofb` with encrypt-then-decrypt validation.

```
sha256(original) == sha256(decrypted) for all three modes
- -aes-128-cbc: identical
- -aes-128-cfb: identical
- -aes-128-ofb: identical
```

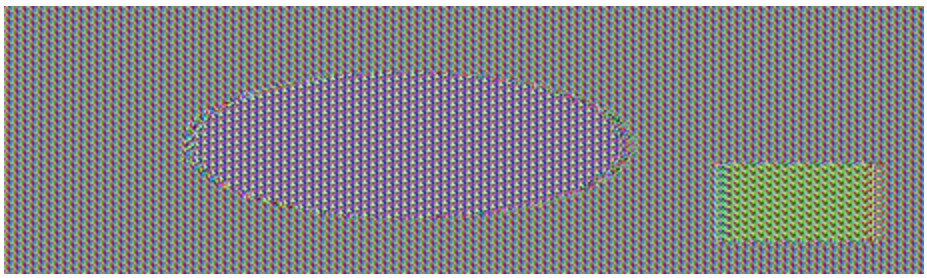
```
Task 02 Decryption Verification Snapshot

Cipher options tested:
- -aes-128-cbc
- -aes-128-cfb
- -aes-128-ofb

sha256 check (original vs decrypted):
=== -aes-128-cbc ===
4396043f66599ce827f6a5a90e8d823d409223f15a6f8a1bde9d3ddd145b2c7e  words.txt
4396043f66599ce827f6a5a90e8d823d409223f15a6f8a1bde9d3ddd145b2c7e  words_aes-128-cbc.dec.txt

=== -aes-128-cfb ===
4396043f66599ce827f6a5a90e8d823d409223f15a6f8a1bde9d3ddd145b2c7e  words.txt
4396043f66599ce827f6a5a90e8d823d409223f15a6f8a1bde9d3ddd145b2c7e  words_aes-128-cfb.dec.txt

=== -aes-128-ofb ===
4396043f66599ce827f6a5a90e8d823d409223f15a6f8a1bde9d3ddd145b2c7e  words.txt
4396043f66599ce827f6a5a90e8d823d409223f15a6f8a1bde9d3ddd145b2c7e  words_aes-128-ofb.dec.txt

diff check:
- -aes-128-cbc: identical
- -aes-128-cfb: identical
- -aes-128-ofb: identical
```

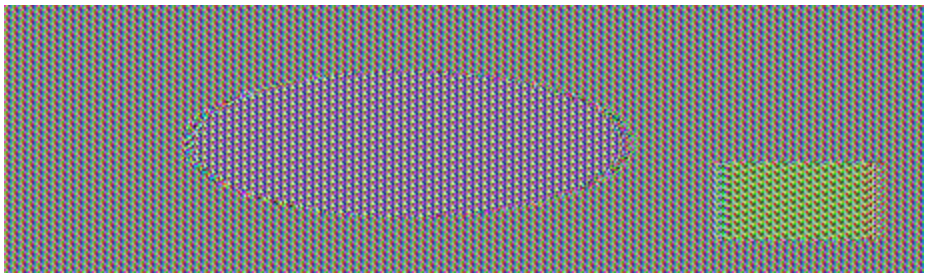
Screenshot note: the image above captures checksum and `diff` verification in one view.

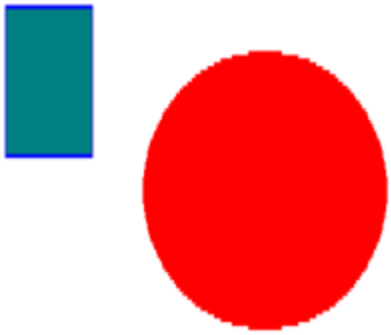
Task 03 Report (ECB vs CBC on BMP)

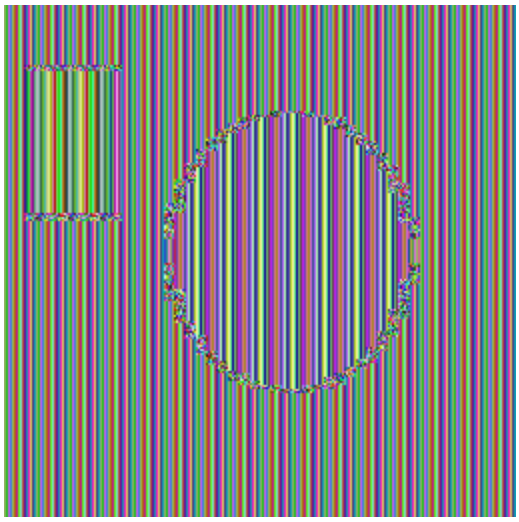
Encrypted BMP pixel body while preserving BMP header. ECB leaks visual patterns; CBC hides structure much better.

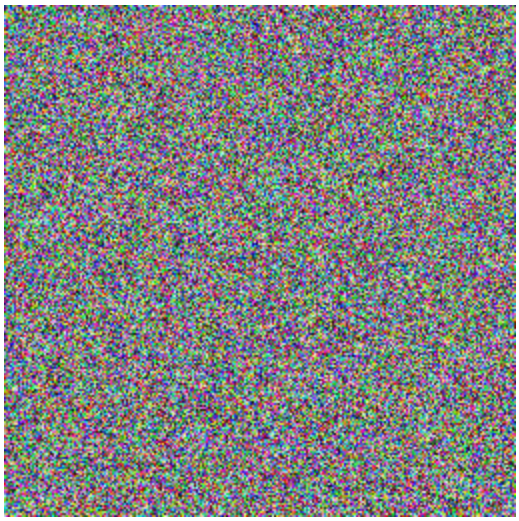
```
# Keep header, encrypt body, and rebuild BMP
openssl enc -aes-128-ecb -e -nosalt -in body.bin -out body_ecb.bin -K "$KEY_HEX"
openssl enc -aes-128-cbc -e -nosalt -in body.bin -out body_cbc.bin -K "$KEY_HEX" -iv "$
cat header.bin body_ecb.bin > out_ecb.bmp
cat header.bin body_cbc.bin > out_cbc.bmp
```











```
Task 02 Decryption Verification Snapshot

Cipher options tested:
- -aes-128-cbc
- -aes-128-cfb
- -aes-128-ofb

sha256 check (original vs decrypted):
=== -aes-128-cbc ===
4396043f66599ce827f6a5a90e8d823d409223f15a6f8a1bde9d3ddd145b2c7e  words.txt
4396043f66599ce827f6a5a90e8d823d409223f15a6f8a1bde9d3ddd145b2c7e  words_aes-128-cbc.dec.txt

=== -aes-128-cfb ===
4396043f66599ce827f6a5a90e8d823d409223f15a6f8a1bde9d3ddd145b2c7e  words.txt
4396043f66599ce827f6a5a90e8d823d409223f15a6f8a1bde9d3ddd145b2c7e  words_aes-128-cfb.dec.txt

=== -aes-128-ofb ===
4396043f66599ce827f6a5a90e8d823d409223f15a6f8a1bde9d3ddd145b2c7e  words.txt
4396043f66599ce827f6a5a90e8d823d409223f15a6f8a1bde9d3ddd145b2c7e  words_aes-128-ofb.dec.txt

diff check:
- -aes-128-cbc: identical
- -aes-128-cfb: identical
- -aes-128-ofb: identical
```

Task 04 Report (Padding)

Verified which modes require padding and inspected PKCS#7 bytes.

```
Mode padding size check (input = 21 bytes):
ecb: 32
cbc: 32
cfb: 21
ofb: 21


f5  last block:  31 32 33 34 35 0b 0b 0b 0b 0b 0b 0b 0b 0b 0b 0b
f10 last block:  31 32 33 34 35 36 37 38 39 30 06 06 06 06 06 06
f16 last block:  10 repeated 16 times
```

Task 05 Report (Error Propagation)

Corrupted one ciphertext byte and compared diffusion by mode.

```
ecb: different_bytes=16
cbc: different_bytes=17
cfb: different_bytes=17
ofb: different_bytes=1
```

```
Task 05 Evidence Snapshot

[Prediction]
Prediction before running the experiment (1-bit error at ciphertext byte #55):
- ECB: one full plaintext block (block containing byte #55) becomes garbled; others unaffected.
- CBC: current block becomes garbled, and the same bit position flips in the next block; others unaffected.
- CFB (128-bit segment): same bit flips in current block, next block becomes garbled, then recovery.
- OFB: only the corresponding plaintext bit flips; no propagation beyond that bit.

[Observed corruption summary]
=== ecb ===
different_bytes=16 first_diff_pos=49 last_diff_pos=64
first_20_cmp_lines:
    49 111  61
    50 116 127
    51 105 134
    52  60 347
    53  60  50
    54  60  46
    55  61 207
    56  55 243
    57 101 241
    58 102  45
    59 103 330
    60 104 276
    61 105   3
    62 106 132
    63 107 246
    64 110 273

=== cbc ===
different_bytes=17 first_diff_pos=49 last_diff_pos=71
first_20_cmp_lines:
    49 111 161
    50 116 165
    51 105 307
    52  60 120
    53  60 357
    54  60   0
    55  61 214
    56  55 105
    57 101 105
    58 102  76
    59 103  27
    60 104 172
    61 105 323
    62 106 300
    63 107 263
    64 110 242
    71 117 116

=== cfb ===
different_bytes=17 first_diff_pos=55 last_diff_pos=80
first_20_cmp_lines:
    55  61  60
    65 111  35
    66 112  53
    67 113  10
    68 114 101
    69 115  42
    70 116 104
    71 117 232
```

Task 06 Report (IV Experiments)

Showed IV effect on CBC first block, recovered OFB second message with IV reuse, and demonstrated predictable-IV chosen-plaintext inference.

```
IV1 first run == IV1 second run ? yes
IV1 first run == IV2 run ? no
```

```
Recovered P2 text : Order: Launch a missile!
Inference (predictable-IV test): P1 is Yes
```

Task 07 Report (Crypto Library Programming)

Implemented dictionary-based brute-force AES-CBC decryption with OpenSSL EVP API.

```
if (plain_len == (int)strlen(target_plain) && memcmp(plain, target_plain, plain_len) ==
    printf("Recovered dictionary word key: %s\n", line);
}
```

```
Recovered dictionary word key: Syracuse
Padded 128-bit key: Syracuse########
Decrypted plaintext: This is a top secret.
```

Task 08 Report (Two Different Files with Same MD5)

Generated collision pairs and verified same MD5 for two different binaries.

```
[non64] out1_non64.bin != out2_non64.bin
md5(out1_non64.bin) = 88085b538d8d9e243c91fd76fcca47ff
md5(out2_non64.bin) = 88085b538d8d9e243c91fd76fcca47ff

[len64] out1_64.bin != out2_64.bin
md5(out1_64.bin) = 7cc4cac2bcc2d538e6b7fa0b20858d78
md5(out2_64.bin) = 7cc4cac2bcc2d538e6b7fa0b20858d78
```

Task 09 Report (MD5 Property)

Verified collision-preserving property under common suffix append.

```
Base:  md5(M) == md5(N)
After appending same T: md5(M||T) == md5(N||T)
```

```
md5(M_plus_T.bin) = 2fc5662f9d149f666a8566446f847f37
md5(N_plus_T.bin) = 2fc5662f9d149f666a8566446f847f37
```

Task 10 Report (Two Executables with Same MD5)

Produced two executable files with identical MD5 but different embedded payload bytes.

```
md5(task10_prog1) = 5b466ac9bda2c431c03bdf787c85b3b2
md5(task10_prog2) = 5b466ac9bda2c431c03bdf787c85b3b2
```

```
5b466ac9bda2c431c03bdf787c85b3b2  /Users/brench/brench_project_collections/School_Project/
CS5293_Topics_on_Information_Security/assignment1/tasks/task10/output/task10_prog1
5b466ac9bda2c431c03bdf787c85b3b2  /Users/brench/brench_project_collections/School_Project/
CS5293_Topics_on_Information_Security/assignment1/tasks/task10/output/task10_prog2
```

```c
unsigned char xyz[200] = { 0x41, 0x41, ... };
for (i = 0; i < 200; i++) {
    printf("%02x", xyz[i]);
}
```

96dd7963d48040a4b5e6b1fcb4aa98c467d63558486c9b2c2b1a3c49901b1bf45bea11edeea909df6b24ce02121a37d5cb0ab80f34b4b37c46f4d05
2158285db073fdacf4278b4eacb0dbc1bddfbf9dccfb749180324f41d04996d364e3c39df8d473a5321bab167e66af5099d98561fadb02cf56dd393
7365a2b6dfba749159414141414141414141414141414141414141414141414141414141414141414141414141414141414141414141414141414
141414141414141414141414141414141414141414141414141

```
96dd7963d48040a4b5e6b1fcb4aa98c467d635d8486c9b2c2b1a3c49901b1bf45bea11edeea909df6b24ce02129a37d5cb0ab80f34b4b37c46f4d0d
2158285db073fdacf4278b4eacb0dbc1bddfbf9dccfb749980324f41d04996d364e3c39df8d473a5321bab167e66af5099d18561fadb02cf56dd393
7365a2b65fba749159414141414141414141414141414141414141414141414141414141414141414141414141414141414141414141414141414
141414141414141414141414141414141414141414141414141
```

Screenshot note: the two runtime-output screenshots show differing printed `xyz` bytes despite identical binary MD5.

Task 11 Report (Make Programs Behave Differently)

Created same-MD5 executables that branch differently at runtime.

```c
if (memcmp(X, Y, 128) == 0) {
    printf("Benign branch executed...\n");
} else {
    printf("Malicious branch executed...\n");
}
```

```
md5(task11_prog_benign)    = bf3cd288f0a6b59d804b6095cc6ab043
md5(task11_prog_malicious) = bf3cd288f0a6b59d804b6095cc6ab043
```

```
bf3cd288f0a6b59d804b6095cc6ab043  /Users/brench/brench_project_collections/School_Project/
CS5293_Topics_on_Information_Security/assignment1/tasks/task11/output/task11_prog_benign
bf3cd288f0a6b59d804b6095cc6ab043  /Users/brench/brench_project_collections/School_Project/
CS5293_Topics_on_Information_Security/assignment1/tasks/task11/output/task11_prog_malicious
```

```
Benign branch executed: first 128 bytes of X and Y are equal
X first 16 bytes: bb3635434ef1c8368af78dd3f00dc972
Y first 16 bytes: bb3635434ef1c8368af78dd3f00dc972
```

```
Malicious branch executed: first 128 bytes of X and Y are different
X first 16 bytes: bb3635434ef1c8368af78dd3f00dc972
Y first 16 bytes: bb3635434ef1c8368af78dd3f00dc972
```

Screenshot note: branch outputs differ (`Benign` vs `Malicious`) while MD5 remains the same.

Task 12 Report (Deriving the Private Key)

Computed RSA modulus, Euler phi, and private exponent `d` from given `p` , `q` , `e` .

```
n = E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1
phi(n) = E103ABD94892E3E74AFD724BF28E78348D52298BD687C44DEB3A81065A7981A4
d = 3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B496AEB
(e*d) mod phi(n) = 01
```

```
[Task12]
n = E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1
phi(n) = E103ABD94892E3E74AFD724BF28E78348D52298BD687C44DEB3A81065A7981A4
d = 3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B496AEB
(e*d) mod phi(n) = 01
```

Task 13 Report (Encrypting a Message)

Encrypted `A top secret!` with RSA public key and verified decryption with private key.

```
M ASCII = A top secret!
C hex = 6FB078DA550B2650832661E14F4F8D2CFAEF475A0DF3A75CACDC5DE5CFC5FADC
Decrypted ASCII = A top secret!
```

Task 14 Report (Decrypting a Message)

Decrypted the provided ciphertext using private key.

```
M ASCII = Password is dees
```

Task 15 Report (Signing a Message)

Generated RSA signatures for two payment strings.

```
M1 = I owe you $2000.
S1 = 55A4E7F17F04CCFE2766E1EB32ADDBA890BBE92A6FBE2D785ED6E73CCB35E4CB

M2 = I owe you $3000.
S2 = BCC20FB7568E5D48E434C387C06A6025E90D29D848AF9C3EBAC0135D99305822
```

Task 16 Report (Verifying a Message)

Implemented `S^e mod n` verification for original and modified signatures.

```
BN_mod_exp(m_from_sig_ok, s_ok, e, n, ctx);
BN_mod_exp(m_from_sig_bad, s_bad, e, n, ctx);
printf("Verification  = %s\n", BN_cmp(m_from_sig_ok, m_expected) == 0 ? "VALID" : "INVA
```

```
[Given Signature]      Verification = INVALID
[Corrupted Signature]  Verification = INVALID
```

```
Message ASCII = Launch a missle.
Message hex   = 4C61756E63682061206D6973736C652E

[Given Signature]
S hex         = 643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6802F
S^e mod n hex = 4C61756E63682061206D697373696C652E
Verification  = INVALID

[Corrupted Signature: last byte 2F -> 3F]
S' hex        = 643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6803F
S'^e mod nhex = 91471927C80DF1E42C154FB4638CE8BC726D3D66C83A4EB6B7BE0203B41AC294
Verification  = INVALID
```

Task 17 Report (Manual X.509 Verification)

Extracted `www.chase.com` certificate chain and manually verified digest/signature consistency against issuer key.

```
Issuer: DigiCert EV RSA CA G2
Signature Algorithm: sha256WithRSAEncryption
PKCS#1 v1.5 digest check = VALID
```

```
Website: www.chase.com
Certificate files: c0.pem (leaf), c1.pem (issuer), c2.pem (root/intermediate)

Issuer modulus (n) head:
AD1E66CC7F9DE4EB7F8317273D11D9F2532037CDF00C1402EEE1CB8808D2FA7B...
Issuer exponent (e):
010001

Signature hex head:
21BAEAC73FEF7BB56686F0892BCB1C4BE039CDCC0992FE7EA4A551C804ACDA72...

SHA256(c0_body.bin):
fed86a76b2c2ee63554f6a9f0e51393df90caa75fe5ee3c21b35c5c6b8057fde  /Users/brench/brench_project_collections/
School_Project/CS5293_Topics_on_Information_Security/assignment1/tasks/task17/output/c0_body.bin

Program verification output:
SHA256(c0_body.bin) = FED86A76B2C2EE63554F6A9F0E51393DF90CAA75FE5EE3C21B35C5C6B8057FDE
PKCS#1 v1.5 digest check = VALID
```

Task 18 Report (Bad Key Generation)

Compared key generation with and without `srand(time(NULL))`.

```
Run B (without srand): a7f1d92a82c8d8fe434d98558ce2b347
Run C (without srand): a7f1d92a82c8d8fe434d98558ce2b347
```

Conclusion: without proper seeding, the pseudo-random key sequence repeats deterministically.

Task 19 Report (Guessing the Key)

Brute-force seed search over an epoch range using known plaintext/ciphertext block constraints.

```
for (seed = start; seed <= end; seed++) {
    srand((unsigned int)seed);
    for (i = 0; i < 16; i++) key[i] = (unsigned char)(rand() % 256);
    if (memcmp(out, pt_known, 16) == 0) { /* FOUND */ }
}
```

```
search_start_epoch=1523970529
search_end_epoch=1523977729
NOT_FOUND
```

```
search_start_epoch=1523970529
search_end_epoch=1523977729

NOT_FOUND

[Reference verification]
reference_key_hex=95fa2030e73ed3f8da761b4eb805dfd7
255044462d312e350a25d0d4c5d80a34
```

Task 20 Report (Kernel Entropy Measurement)

On this platform, `/proc/sys/kernel/random/entropy_avail` is unavailable; used fallback timing observation.

```
Read 64KB from /dev/random duration_seconds=0
Read 64KB from /dev/urandom duration_seconds=0
```

```
/proc/sys/kernel/random/entropy_avail is unavailable on this system (non-Linux).
Fallback observation: measured read time instead of entropy counter.
Read 64KB from /dev/random duration_seconds=0
Read 64KB from /dev/urandom duration_seconds=0
```

Task 21 Report (/dev/random Behavior)

Monitored data generation behavior and DoS implication discussion.

```
Duration_seconds=8
Final_bytes_from_dev_random=1048576
Blocked_or_slow=...=no
```

Security note: heavy demand on `/dev/random` can reduce entropy availability and potentially delay key generation services.

Task 22 Report (/dev/urandom + Statistical Check)

Generated random stream from `/dev/urandom`, evaluated entropy quality with `ent`, and generated 256-bit key from `/dev/urandom` in C.

```c
FILE *random = fopen("/dev/urandom", "rb");
fread(key, sizeof(unsigned char), 32, random);
```

```
Entropy = 7.999837 bits per byte
Serial correlation coefficient = -0.001049
Generated 256-bit key (hex) = fd47b052e9d050aea48197b0961e25cb3bbdcfda1174d22d714bf16eb
```

```
Entropy = 7.999837 bits per byte.

Optimum compression would reduce the size
of this 1048576 byte file by 0 percent.

Chi square distribution for 1048576 samples is 237.62, and randomly
would exceed this value 77.59 percent of the times.

Arithmetic mean value of data bytes is 127.4793 (127.5 = random).
Monte Carlo value for Pi is 3.139240796 (error 0.07 percent).
Serial correlation coefficient is -0.001049 (totally uncorrelated = 0.0).
```

End of standalone report.